LECTURE 9

Lexical Scope, Closures & the “this” Keyword

How JavaScript resolves variables, remembers scope, and decides what “this” points to — in regular functions vs. arrow functions.

`var` • `const` • `function` • `=>`

AGENDA

What we'll cover today

01	**Lexical Scope**	How a function's position in the code decides which variables it can see.
02	**Closures**	How a function keeps access to its birth scope even after that scope has finished running.
03	**“this” — the basics**	Default binding in the browser vs. Node, and how it changes at runtime.
04	**“this”: Normal vs. Arrow**	13 worked examples comparing how each function type resolves “this”.

Scope is decided by where you write the code

Definition

Lexical scope means a function's scope is fixed by its physical placement in the source code — not by where or how it is called.

Key idea

A function can always reach variables declared in its outer (parent) scopes, because of where it sits in the code.

```
function outer() {
  const outerVar = "I am outer";

  function inner() {
    console.log(outerVar);
    // ✅ accessible: lexical scope
  }

  inner();
}

outer();
```

Why inner() can see outerVar

```
function outer() {
  const outerVar = "I am outer";

  function inner() {
    console.log(outerVar);
  }

  inner();
}

outer();
```

OUTPUT

```
I am outer
```

How to read it

- inner() is physically written inside outer(), so it sits one level deeper in the scope chain.
- JavaScript looks up outerVar first in inner()'s own scope, doesn't find it, then walks outward to outer()'s scope.
- This lookup path is fixed at write-time — it never depends on who calls inner().

A function that remembers where it came from

↻ **Definition**

A closure is a function that keeps “remembering” the variables from its lexical scope, even after the outer function has already finished executing.

Key idea

Closures let a function keep access to its scope even when it runs later, outside of its original context.

```
function outer() {
  const outerVar = "I am outer";
  return function inner() {
    console.log(outerVar);
    // still accessible: closure
  };
}

const innerFn = outer();
// outer() has finished running
innerFn();  // "I am outer"
```

outer() is done — but inner() still remembers

```
function outer() {
  const outerVar = "I am outer";
  return function inner() {
    console.log(outerVar);
  };
}

const innerFn = outer();
innerFn();
```

OUTPUT

```
I am outer
```

How to read it

- outer() runs, returns inner, and then finishes — normally its local variables would be gone.
- But inner() was defined inside outer(), so it keeps a live link to outerVar.
- That preserved link is the closure: innerFn() still logs “I am outer” long after outer() has returned.

“this” always points to an object

If there’s no object at the call site, the runtime supplies a default one — and browsers and Node disagree on what that default is.

BROWSER ENGINE

Window object

No enclosing object → “this” falls back to the global Window object.

NODE.JS ENVIRONMENT

Empty object {}

At the top level of a module, “this” resolves to an empty object, not the global object.

Same script, two different runtimes

```
var a = 6;
this.b = 8;
console.log(this);
this.a = 12;
console.log(a);
console.log(this.a);
console.log(this);
```

OUTPUT

```
{ Window }
12
12
{ Window }
```

BROWSER

OUTPUT

```
{ b: 8 }
6
12
{ b:8, a:12 }
```

NODE.JS

One rule explains every example ahead

```
function () { }
```

Gets its own “this”, decided by how it is called (the call site). This is called implicit / dynamic binding.

```
() => { }
```

Has no “this” of its own. It lexically inherits “this” from whatever scope it was written inside — its parent.

EXAMPLE 1 · REGULAR FUNCTION AS A METHOD

Implicit binding: “this” = the calling object

```
const A = {
  name: "Gaurav",
  callback: function () {
    console.log(this.name);
  }
};

A.callback();
```

OUTPUT

Gaurav

- callback is a regular function, so its “this” is set by the call site: A.callback().
- Because A is the object left of the dot, “this” becomes A — so this.name is “Gaurav”.

Arrow functions skip implicit binding entirely

```
const A = {
  name: "Gaurav",
  callback: () => {
    console.log(this.name);
  }
};

A.callback();
```

OUTPUT

```
undefined
(this = Window object)
```

- callback is an arrow function, so it has no “this” of its own — it looks outward.
- It was written at the top level, so its parent scope’s “this” is the Window object, not A.
- this.name on Window is blank/undefined, since Window has no name property yet.

EXAMPLE 3 · ARROW FUNCTION READS A GLOBAL PROPERTY

Set this.name globally → arrow method sees it

```
console.log(this);
this.name = "Raj";

const A = {
  name: "Gaurav",
  callback: () => {
    console.log(this.name);
  }
};

A.callback();
```

OUTPUT

```
{ Window object }
Raj
```

- this.name = "Raj" at the top level adds a name property directly onto the Window object.
- The arrow callback's "this" still points to Window (its lexical parent) — so this.name reads "Raj", ignoring A.name.

EXAMPLE 4 · ARROW FUNCTION LOGGING “THIS” DIRECTLY

The arrow method itself resolves to Window

```
const A = {
  name: "Gaurav",
  callback: () => {
    console.log(this);
  }
};

A.callback();
```

OUTPUT

```
Window Object
```

- Even called as A.callback(), the arrow function never rebinds “this” to A.
- “this” in an arrow function always points to the object of its parent (enclosing) function — here, the top level, so Window.

EXAMPLE 5 · ARROW FUNCTION NESTED INSIDE A REGULAR METHOD

Arrow inherits “this” from its enclosing function

```
const A = {
  name: "Gaurav",
  callback: function () {
    arrow = () => {
      console.log(this);
    };
    arrow();
  }
};

A.callback();
```

OUTPUT

```
A Object
```

- callback is a regular function, so calling A.callback() binds its “this” to A (implicit binding).
- arrow is defined lexically inside callback, so it inherits that same “this” — the A object.
- This is the general rule: an arrow function’s “this” = its nearest enclosing (non-arrow) function’s “this”.

Nested arrow → this.name resolves through the parent

```
const A = {
  name: "Gaurav",
  callback: function () {
    arrow = () => {
      console.log(this.name);
    };
    arrow();
  }
};

A.callback();
```

OUTPUT

```
Gaurav
```

- Same structure as Example 5: arrow borrows “this” from callback, which is bound to A.
- So this.name inside arrow reads A.name → “Gaurav”.

EXAMPLE 7 · SIDE-BY-SIDE COMPARISON

Regular method vs. arrow method, same object

```
this.name = "Gaurav Raj";      // set globally first

const A = {
  name: "Gaurav",
  callback1: function () { console.log(this.name); },    // regular
  callback2: () => { console.log(this.name); }           // arrow
};

A.callback1();
A.callback2();
```

OUTPUT

callback1() → Gaurav

OUTPUT

callback2() → Gaurav Raj

callback1 (regular) is implicitly bound to A → reads A.name. callback2 (arrow) inherits the outer/global “this”, which now has name = “Gaurav Raj”.

EXAMPLE 8 · REGULAR FUNCTION NESTED INSIDE A METHOD

A plain nested function does NOT inherit “this”

```
this.name = "Gaurav Raj";

const A = {
  name: "Gaurav",
  callback1: function () {
    function abc() {
      console.log(this.name);
    }
    abc();
  }
};

A.callback1();
```

OUTPUT

```
Gaurav Raj
```

- abc() is a regular function, so it gets its own “this” decided by how it’s called: abc() — a plain call, not A.abc().
- A plain call defaults “this” to the global object, NOT to A — abc does not inherit its parent function’s object.
- That’s why this.name reads the global “Gaurav Raj”, not A’s “Gaurav”.

EXAMPLE 9 · ARROW FUNCTION NESTED INSIDE A METHOD

The arrow version DOES inherit “this” from the parent

```
this.name = "Gaurav Raj";

const A = {
  name: "Gaurav",
  callback1: function () {
    abc = () => {
      console.log(this.name);
    };
    abc();
  }
};

A.callback1();
```

OUTPUT

```
Gaurav
```

- callback1 is a regular function bound to A via implicit binding (A.callback1()).
- abc is an arrow function, so it lexically inherits “this” from callback1 — which is A.
- Contrast with Example 8: swapping function abc(){} for an arrow flips the result from global to A.

Call-site binding travels with the function

```
var roll = 7;

function callnotchild() {
  console.log(this.roll);
}

const A = {
  roll: 1,
  parent: function (notchild) {
    notchild();
  }
};

A.parent(callnotchild);
```

OUTPUT

```
7
```

- callnotchild is invoked as a plain call (notchild()) inside parent — not as A.notchild().
- So its “this” defaults to the global object, where roll is 7 — A.roll (1) is never used.
- Passing a function around doesn’t carry its “home” object with it; only the call site matters for regular functions.

Arrow callbacks keep the “this” they were born with

```
var roll = 17;

callchild = () => {
  console.log(this.roll);
};

const A = {
  roll: 1,
  parent: function (child) {
    child();
  }
};

callchild();          // direct call
A.parent(callchild);  // passed into A
```

OUTPUT

callchild() → 17

OUTPUT

A.parent(callchild) → 17

- callchild is an arrow function defined at the top level, so its “this” is permanently the global object.
- Even when passed into A.parent and invoked from inside a method, the result stays 17 — arrow functions ignore how or where they’re called.

Regular functions also expose an arguments object

```
var roll = 17;

callchild = () => { console.log(this.roll); };

const A = {
  roll: 1,
  parent: function (child) {
    child();
    console.log(arguments);
    arguments[0]();
    console.log(arguments[2]);
  }
};

A.parent(callchild, 5, 7);
```

OUTPUT

```
17
Arguments(3)
17
7
```

- parent is a regular function, so it receives an array-like arguments object of everything passed to it: [callchild, 5, 7].
- arguments[0]() calls callchild again — still 17, since it's an arrow function bound to the global “this”.
- arguments[2] is simply the third argument passed in: 7.

EXAMPLE 13 · METHOD CHAINING

“return this” lets methods chain together

```
var calculator = {
  total: 0,
  addition(p)       { this.total += p; return this; },
  subtraction(p)    { this.total -= p; return this; },
  multiplication(p) { this.total *= p; return this; },
  division(p)       { this.total /= p; return this; }
};

var result = calculator
  .addition(13)
  .subtraction(7)
  .multiplication(5)
  .division(2);

console.log(result.total);
```

OUTPUT

```
result.total → 15
```

How chaining works

- Every method mutates this.total, then returns this — the calculator object itself.
- Because each call returns the same object, the next method can be called immediately with a dot: no temp variables needed.
- (13 − 7) × 5 ÷ 2 = 15, matching this.total at the end of the chain.

SUMMARY

Normal function vs. arrow function — cheat sheet

	`function () {}`	`() => {}`
Own “this”?	Yes — set at call time	No — none of its own
Where “this” comes from	The object left of the dot at the call site	Lexically, from the nearest enclosing function
Called as obj.method()	“this” = obj	“this” = obj’s enclosing scope (often global)
Called as a plain callback	“this” defaults to global object	“this” stays whatever it was at definition
Has its own arguments?	Yes	No — inherits parent’s



Thank You

```
console.log("Lecture 9 complete");
```